Doc. no. P0619R1
Date: 2017-03-19
Project: Programming Language C++
Audience: Evolution Working Group
Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Stephan T. Lavavej <stl@microsoft.com>
Tomasz Kamiński <tomaszkam at gmail dot com>

Reviewing Deprecated Facilities of C++17 for C++20

Table of Contents

Revision History

Revision 0

Original version of the paper for the 2017 post-Kona mailing.

Revision 0

First update for the 2017 pre-Toronto mailing:

- cross-reference library issues: 2155
- weak recommendation to undeprecate `strstreamS`

1 Introduction

This paper evaluates all the existing deprecated facilities in the C++17 standard, and recommends a subset as candidates for removal in C++20.

2 Stating the problem

With the release of a new C++ standard, we get an opportunity to revisit the features identified for deprecation, and consider if we are prepared to clear any out yet, either by removing completely from the standard, or by reversing the deprecation decision and restoring the feature to full service.

In an ideal world, the start of every release cycle would cleanse the list of deprecated features entirely, allowing the language and library to evolve cleanly without holding too much deadweight. In practice, C++ has some long-term deprecated facilities that are difficult to remove, and equally difficult to rehabilitate. Also, with the three year release cadence for the C++ standard, we will often be considering removal of features whose deprecated status has barely reached print.

The benefits of making the choice to remove early is that we will get the most experience we can from the bleeding-edge adopters whether a particular removal is more problematic than expected - but even this data point is limited, as bleeding-edge adopters typically have less reliance on deprecated features, eagerly adopting the newer replacement facilities.

We have precedent that Core language features are good targets for removal, typically taking two standard cycles to remove a deprecated feature, although often prepared to entirely remove a feature even without a period of deprecation, if the cause is strong enough.

The library experience has been mixed, with no desire to remove anything without a period of deprecation (other than `gets`) and no precedent prior to C++17 for actually removing deprecated features.

The precedent set for the library in C++17 seems to be keener to clear out the old code more quickly though, even removing features deprecated as recently as C++14. Accordingly, this paper will be fairly aggressive in its attempt to clear out old libraries. This is perhaps more reasonable for the library clauses than the core language, as the Zombie Names clause, **20.5.4.3.1 [zombie.names]** allows vendors to continue shipping features long after they have left the standard, as long as their existing customers rely on them.

This paper makes no attempt to offer proposals for removing features other than those deprecated in Annex D, nor does it attempt to identify new candidates for deprecation.

3 Proposed Solution

We will review each deprecated facility, and make a strong and a weak recommendation. The strong recommendation is the preferred direction of the authors, who lean towards early removal. There will also be a weak recommendation, which is an alternative proposal for the evolution groups to consider, if the strong recommendation does not find favor. Finally, wording is generally provided for both the strong and weak recommendations, which will be collated into a unified Proposed Wording section once the various recommendations have been given direction by the corresponding evolution group.

All proposed wording is relative to the expected C++17 DIS, including the revised clause numbers. However, it will be updated to properly reflect the final document in the pre-Toronto mailing, incorporating any feedback we accrue in the meantime.

D.1 Redeclaration of static constexpr data members [depr.static_constexpr]

Redundant `constexpr` declarations are a relatively recent feature of modern C++, as the `constexpr` facility was not introduced until C++11. That said, the redundant redeclaration was not deprecated until C++17, so there has not yet been much time for user code to respond to the deprecation notice.

The feature seems relatively small and harmless; it is not clear that there is a huge advantage to removing it immediately from C++20. That said, there is also an argument to be made for cleanliness in the standard and prompt removal of deprecated features. If we do wish to consider the removal of redundant `constexpr` declarations, it would be best to do so early, so that there is an opportunity for early adopters to shout if the feature is relied on more heavily than expected.

Strong recommendation: take no action yet, consider again for C++23.

No change.

Weak recommendation: remove this facility from C++20.

6.1 Declarations and definitions [basic.def]

2. A declaration is a *definition* unless
    1. — it declares a function without specifying the function's body (11.4),
    2. — it contains the `extern` specifier (10.1.1) or a *linkage-specification*[26] (10.5) and neither an *initializer* nor a *function-body*,
    3. — it declares a non-inline static data member in a class definition (12.2, 12.2.3),
    4. ~~— it declares a static data member outside a class definition and the variable was defined within the class~~

~~with the `constexpr` specifier (this usage is deprecated; see D.1),~~

5. — it is a class name declaration (12.1),
6. — it is an *opaque-enum-declaration* (10.2),
7. — it is a *template-parameter* (17.1),
8. — it is a *parameter-declaration* (11.3.5) in a function declarator that is not the *declarator* of a *function-definition*,
9. — it is a `typedef` declaration (10.1.3),
10. — it is an *alias-declaration* (10.1.3),
11. — it is a *using-declaration* (10.3.3),
12. — it is a *deduction-guide* (17.9),
13. — it is a *static_assert-declaration* (Clause 10),
14. — it is an *attribute-declaration* (Clause 10),
15. — it is an *empty-declaration* (Clause 10),
16. — it is a *using-directive* (10.3.4),
17. — it is an explicit instantiation declaration (17.7.2), or
18. — it is an explicit specialization (17.7.3) whose *declaration* is not a definition.

12.2.3.2 Static data members [class.static.data]

3. If a non-volatile non-inline `const` static data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (8.20). The member shall still be defined in a namespace scope if it is odr-used (6.2) in the program and the namespace scope definition shall not contain an *initializer*. An inline static data member may be defined in the class definition and may specify a *brace-or-equal-initializer*. ~~If the member is declared with the constexpr specifier, it may be redeclared in namespace scope with no initializer (this usage is deprecated; see D.1).~~ Declarations of other static data members shall not specify a *brace-or-equal-initializer*.

~~**D.1 Redeclaration of static constexpr data members [depr.static_constexpr]**~~

1. ~~For compatibility with prior C++ International Standards, a constexpr static data member may be redundantly redeclared outside the class with no initializer. This usage is deprecated. [*Example:*~~

```
struct A {
  static constexpr int n = 5; // definition (declaration in C++ 2014)
};
constexpr int A::n; // redundant declaration (definition in C++ 2014)
```

~~*— end example*]~~

Draft compatibility note for Annex C.

We do not anticipate any impact on library wording by this removal.

D.2 Implicit declaration of copy functions [depr.impldec]

This feature was deprecated towards the end of the C++11 development cycle, and was heavily relied on prior to that. That said, many of the deprecated features are implicitly deleted if any move operations are declared, and modern users are increasingly familiar with this idiom, and may find the older deprecated behavior jarring. It is expected that compilers will give good warnings for code that breaks with the removal of this feature, much as they advise about its impending demise with deprecation warnings today.

Strong recommendation: take decisive action early in the C++20 development cycle, so early adopters can shake out the reamaining cost of updating old code. Note that this would be both an API and ABI breaking change, and it would be good to set that precedent early if we wish to allow such breakage in C++20.

Consider 11.4.2 [dcl.fct.def.default]p5, explicitly defaulting a copy ctor after a user-declaration in the class definition. If this definition were to become deleted, the program would become ill-formed, and that would be bad. Ensure we have wording to cover this case.

15.8.1 Copy/move constructors [class.copy.ctor]

6. If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor, ~~or~~ move assignment operator, copy assignment operator, or destructor, the implicitly declared copy constructor is defined as deleted; otherwise, it is defined as defaulted

(11.4). ~~The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor.~~

15.8.2 Copy/move assignment operator [class.copy.assign]

2. If the class definition does not explicitly declare a copy assignment operator, one is declared *implicitly*. If the class definition declares a move constructor, ~~or~~ move assignment operator, copy constructor, or destructor, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defined as defaulted (11.4). ~~The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor.~~ The implicitly-declared copy assignment operator for a class `X` will have the form...

~~**D.2 Implicit declaration of copy functions [depr.impldec]**~~

1. ~~The implicit definition of a copy constructor as defaulted is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor. The implicit definition of a copy assignment operator as defaulted is deprecated if the class has a user-declared copy constructor or a user-declared destructor (15.4, 15.8). In a future revision of this International Standard, these implicit definitions could become deleted (11.4).~~

> Audit standard for examples like 10.1.7.2 [dcl.type.simple]p5 that rely on implicit copies that will now be deleted.

> Draft compatibility note for Annex C.

Weak recommendation: take no action now, and plan to remove this feature in the next revision of the standard that will permit both ABI and API breakage.

No change.

D.3 Deprecated exception specifications [depr.except.spec]

The deprecated exception specification `throw()` was retained in C++17, when other exception specifications were removed, to retain compatibility for the only form that saw widespread use. Its impact on the remaining standard is minimal, so it costs little to retain it for another iteration, giving users a reasonable amout of time to complete their transition, and retaining a viable conversion path from C++03 directly to C++20. It is worth noting that the feature will have been deprecated for most of a decade when C++20 is published though.

Strong recommendation: take no action now, and plan to remove this feature in C++23.

No change.

Weak recommendation: remove the final traces of the deprecated exception specification syntax:

18.4 Exception specifications [except.spec]

1. The predicate indicating whether a function cannot exit via an exception is called the *exception specification* of the function. If the predicate is false, the function has a *potentially-throwing exception specification*, otherwise it has a *non-throwing exception specification*. The exception specification is either defined implicitly, or defined explicitly by using a *noexcept-specifier* as a suffix of a function declarator (11.3.5).

```
noexcept-specifier :
  noexcept ( constant-expression )
  noexcept
  throw ( )
```

2. In a *noexcept-specifier*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool` (8.20); that constant expression is the exception specification of the function type in which the *noexcept-specifier* appears. A (token that follows `noexcept` is part of the *noexcept-specifier* and does not commence an initializer (11.6). The *noexcept-specifier* `noexcept` without a *constant-expression* is equivalent to the *noexcept-specifier* `noexcept(true)`. ~~The *noexcept-specifier* `throw()` is deprecated (D.3), and equivalent to the *noexcept-specifier* `noexcept(true)`.~~

11. [*Example:*

```
struct A {
  A(int = (A(5), 0)) noexcept;
  A(const A&) noexcept;
  A(A&&) noexcept;
```

```
  ~A();
};
struct B {
  B() throw()noexcept;
  B(const B&) = default;  // implicit exception specification is noexcept(true)
  B(B&&, int = (throw Y(), 0)) noexcept;
  ~B() noexcept(false);
};
int n = 7;
struct D : public A, public B {
  int * p = new int[n];
  // D::D() potentially-throwing, as the new operator may throw bad_alloc or bad_array_new_length
  // D::D(const D&) non-throwing
  // D::D(D&&) potentially-throwing, as the default argument for B's constructor may throw
  // D::~D() potentially-throwing
};
```

Furthermore, if `A::~A()` were virtual, the program would be ill-formed since a function that overrides a virtual function from a base class shall not have a potentially-throwing exception specification if the base class function has a non-throwing exception specification. — *end example*]

~~D.3 Deprecated exception specifications [depr.except.spec]~~

1. ~~The *noexcept-specifier* `throw()` is deprecated.~~

   Draft compatibility note for Annex C.

D.4 C++ standard library headers [depr.cpp.headers]

The deprecated compatibility headers for C++ are mostly vacuous, merely redirecting to other headers, or defining macros that are alreadty reserved to the implementation. By aliasing the C headers providing macros to emulate the language support for keywords in C++, there is no real value intended by supplying a C++ form of these headers. There should be minimal harm in removing these headers, and it would remove a possible risk of confusion, with users trying to understand if this is some clever compatibility story for mixed C/C++ projects.

LWG Issue 2155 specifically talks about removing `__bool_true_false_are_defined`

Strong recommendation: remove the final traces of the deprecated compatibility headers:

20.5.1.2 Headers [headers]

Table 17 — C++ headers for C library facilities

`<cassert>`	`<cinttypes>`	`<csignal>`	`<cstdio>`	`<cwchar>`
~~`<ccomplex>`~~	~~`<ciso646>`~~	~~`<cstdalign>`~~	`<cstdlib>`	`<cwctype>`
`<cctype>`	`<climits>`	`<cstdarg>`	`<cstring>`	
`<cerrno>`	`<clocale>`	~~`<cstdbool>`~~	~~`<ctgmath>`~~	
`<cfenv>`	`<cmath>`	`<cstddef>`	`<ctime>`	
`<cfloat>`	`<csetjmp>`	`<cstdint>`	`<cuchar>`	

20.5.1.3 Freestanding implementations [compliance]

Table 19 — C++ headers for freestanding implementations

	subclause	**header**
		~~`<ciso646>`~~
21.2	Types	`<cstddef>`
21.3	Implementation properties	`<cfloat>` `<limits>` `<climits>`
21.4	Integer types	`<cstdint>`
21.5	Start and termination	`<cstdlib>`
21.6	Dynamic memory management	`<new>`
21.7	Type identification	`<typeinfo>`
21.8	Exception handling	`<exception>`
21.9	Initializer lists	`<initializer_list>`
21.10	Other runtime support	`<cstdarg>`
23.15	Type traits	`<type_traits>`

20.5.4.3.1 Zombie names [zombie.names]

2. The header names <ccomplex>, <ciso646>, <cstdalign>, <cstdbool>, and <ctgmath> are reserved for previous standardization.

C.5.1 Modifications to headers [diff.mods.to.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in D.4, but their use is deprecated in C++.
2. There are no C++ headers for the C headers <stdatomic.h>, <stdnoreturn.h>, and <threads.h>, nor are the C headers themselves part of C++.
3. The headers <ccomplex> (29.5.11) and <ctgmath> (29.9.6), as well as their corresponding C headers <complex.h> and <tgmath.h>, do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.
4. The headers <ciso646>, <cstdalign> (21.10.4), and <cstdbool> (21.10.3) are meaningless in C++. Use of the C++ headers <ccomplex>, <cstdalign>, <cstdbool>, and <ctgmath> is deprecated (D.4).

C.5.2.4 Header <iso646.h> [diff.header.iso646.h]

1. The tokens and, and_eq, bitand, bitor, compl, not_eq, not, or, or_eq, xor, and xor_eq are keywords in this International Standard (5.11). They do not appear as macro names defined in <ciso646>.

C.5.2.5 Header <stdalign.h> [diff.header.stdalign.h]

1. The token alignas is a keyword in this International Standard (5.11). It does not appear as a macro name defined in <cstdalign> (21.10.4).

C.5.2.6 Header <stdbool.h> [diff.header.stdbool.h]

1. The tokens bool, true, and false are keywords in this International Standard (5.11). They do not appear as macro names defined in <cstdbool> (21.10.3).

D.4 C++ standard library headers [depr.cpp.headers]

1. For compatibility with prior C++ International Standards, the C++ standard library provides headers <ccomplex> (D.4.1), <cstdalign> (D.4.2), <cstdbool> (D.4.3), and <ctgmath> (D.4.4). The use of these headers is deprecated.

D.4.1 Header <ccomplex> synopsis [depr.ccomplex.syn]

```
#include <complex>
```

1. The header <ccomplex> behaves as if it simply includes the header <complex> (29.5.1).

D.4.2 Header <cstdalign> synopsis [depr.cstdalign.syn]

```
#define __alignas_is_defined 1
```

1. The contents of the header <cstdalign> are the same as the C standard library header <stdalign.h>, with the following changes: The header and the header <stdalign.h> shall not define a macro named alignas.
See also: ISO C 7.15.

D.4.3 Header <cstdbool> synopsis [depr.cstdbool.syn]

```
#define __bool_true_false_are_defined 1
```

1. 1 The contents of the header <cstdbool> are the same as the C standard library header <stdbool.h>, with the following changes: The header <cstdbool> and the header <stdbool.h> shall not define macros named bool, true, or false.
See also: ISO C 7.18.

D.4.4 Header <ctgmath> synopsis [depr.ctgmath.syn]

~~#include <complex>~~
~~#include <cmath>~~

1. ~~The header <ctgmath> simply includes the headers <complex> (29.5.1) and <cmath> (29.9.1).~~
2. ~~[*Note:* The overloads provided in C by type-generic macros are already provided in <complex> and <cmath> by "sufficient" additional overloads. — end note]~~

Draft compatibility note for Annex C.

Weak recommendation: take no action now, and plan to remove this feature in C++23.

No change.

D.5 C standard library headers [depr.c.headers]

The basic C library headers are an essential compatibility feature, and not going anywhere anytime soon. However, there are certain C++ specific counterparts that do not bring value, particularly where the corresponding C header's job is to supply macros that masquerade as keywords already present in the C++ langauge.

One possibility to be more aggressive here, following the decision to not adopt *all* C11 headers as part of the C++ mapping to C, would be to remove those same C headers from the subset that must be shipped with a C++ compiler. This would not prevent those headers being supplied, as part of a full C implementation, but would indicate that they have no value to a C++ system.

Finally, it seems clear that the C headers will be retained essentially forever, as a vital compatibility layer with C and POSIX. It may be worth undeprecating the headers, and finding a home for them as a compatibility layer in the main standard. It is also possible that we will want to explore a different approach in the future once modules are part of C++, and the library is updated to properly take advantage of the new language feature. Therefore, we make no recommendation to undeprecate these headers for C++20, but will keep looking into this space for C++23.

strong recommendation: Undeprecate the remaining **[depr.c.headers]** and move it directly into 20.5.5.2 **[res.on.headers]**.

20.5.5.2 Headers [res.on.headers]

1. A C++ header may include other C++ headers. A C++ header shall provide the declarations and definitions that appear in its synopsis. A C++ header shown in its synopsis as including other C++ headers shall provide the declarations and definitions that appear in the synopses of those other headers.
2. Certain types and macros are defined in more than one header. Every such entity shall be defined such that any header that defines it may be included after any other header that also defines it (6.2).
3. ~~The C standard library headers (D.4) shall include only their corresponding C++ standard library header, as described in 20.5.1.2.~~

   Actually, probably want to merge this into **20.5.1.2 Headers [headers]**. Also, fix up cross-references to list stable labels.

20.5.5.2.1 C standard library headers [c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers shown in Table 141.

   Table 141 — C headers

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
`<complex.h>`	`<iso646.h>`	`<stdalign.h>`	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	`<stdbool.h>`	`<tgmath.h>`	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

2. The header `<complex.h>` behaves as if it simply includes the header `<complex>`. The header `<tgmath.h>` behaves as if it simply includes the headers `<complex>` and `<cmath>`.
3. Every other C header, each of which has a name of the form `name.h`, behaves as if each name placed in the standard library namespace by the corresponding cname header is placed within the global namespace scope, except for the functions described in 29.9.5, the declaration of std::byte (21.2.1), and the functions and function templates described in 21.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.3.6) of the namespace std and are then injected into the global namespace scope by explicit

using-declarations (10.3.3).

4. [*Example:* The header <cstdlib> assuredly provides its declarations and definitions within the namespace std. It may also provide these names within the global namespace. The header <stdlib.h> assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace std. — *end example*]

D.5 C standard library headers [depr.c.headers]

1. 1 For compatibility with the C standard library, the C++ standard library provides the *C headers shown* in Table 141.

   Table 141 — C headers

<assert.h>	<inttypes.h>	<signal.h>	<stdio.h>	<wchar.h>
<complex.h>	<iso646.h>	<stdalign.h>	<stdlib.h>	<wctype.h>
<ctype.h>	<limits.h>	<stdarg.h>	<string.h>	
<errno.h>	<locale.h>	<stdbool.h>	<tgmath.h>	
<fenv.h>	<math.h>	<stddef.h>	<time.h>	
<float.h>	<setjmp.h>	<stdint.h>	<uchar.h>	

2. The header <complex.h> behaves as if it simply includes the header <ccomplex> The header <tgmath.h> behaves as if it simply includes the header <ctgmath>.
3. Every other C header, each of which has a name of the form name.h, behaves as if each name placed in the standard library namespace by the corresponding *cname* header is placed within the global namespace scope, except for the functions described in 29.9.5, the declaration of std::byte (21.2.1), and the functions and function templates described in 21.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.3.6) of the namespace std and are then injected into the global namespace scope by explicit using-declarations (10.3.3).
4. [*Example:* The header <cstdlib> assuredly provides its declarations and definitions within the namespace std. It may also provide these names within the global namespace. The header <stdlib.h> assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace std. — *end example*]

Weak recommendation: In addition to tbe above, also remove the corresponding C headers from the C++ standard, much as we have no corresponding <stdatomic.h>, <stdnoreturn.h>, or <threads.h>, headers.

As above, but with the following tweaks:

20.5.5.2.1 C standard library headers [c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers shown in Table 141.

   Table 141 — C headers

<assert.h>	<inttypes.h>	<signal.h>	<stdio.h>	<wchar.h>
<complex.h>	<iso646.h>	<stdalign.h>	<stdlib.h>	<wctype.h>
<ctype.h>	<limits.h>	<stdarg.h>	<string.h>	
<errno.h>	<locale.h>	<stdbool.h>	<tgmath.h>	
<fenv.h>	<math.h>	<stddef.h>	<time.h>	
<float.h>	<setjmp.h>	<stdint.h>	<uchar.h>	

2. The header <complex.h> behaves as if it simply includes the header <complex>. The header <tgmath.h> behaves as if it simply includes the headers <complex> and <cmath>.

C.5.1 Modifications to headers [diff.mods.to.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in D.5, but their use is deprecated in C++.
2. There are no C++ headers for the C headers <complex.h>, <stdatomic.h>, <iso646.h>, <stdalign.h>, <stdbool.h>, <stdnoreturn.h>, <tgmath.h>, and <threads.h>, nor are the C headers themselves part of C++.
3. The headers <ccomplex> (29.5.11) and <ctgmath> (29.9.6), as well as their corresponding C headers <complex.h> and <tgmath.h>, do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.
4. The headers <ciso646>, <cstdalign> (21.10.4), and <cstdbool> (21.10.3) are meaningless in C++. Use of the C++ headers <ccomplex>, <cstdalign>, <cstdbool>, and <ctgmath> is deprecated (D.4).

Draft compatibility note for Annex C.

D.6 `char*` streams [depr.str.strstreams]

The `char*` streams were provided, pre-deprecated, in C++98 and have been considered for removal before. The underlying principle of not removing them until a suitable replacement is available still holds, so there should be nothing further to do at this point. However, it is looking increasingly likely that any future replacement facility will be part of the `std2` library initiative, providing a wholesale replacement for the current iostreams facility. If we believe that the future replacement is more likely to come from this direction, it would be better to integrate this facility back into the main standard, as it would clearly not be due a replacement within namespace `std` at that point.

Strong recommendation: take no action.

No change.

Weak recommendation: Undeprecate the `char*` streams.

Wording to follow on demand, moving subclause D.6 into clause 30.

D.7 `uncaught_exception` [depr.uncaught]

This function is a remnant of the early attempts to implement an exception-aware API. It turned out to be surprisingly hard to specify, not least because it was not entirely obvious at the time that more than one exception may be simultaneously active in a single thread.

The function seems harmless, and takes up little space in the standard, so there is no immediate rush to remove it. As a low level (language support) part of the free-standing library, it is likely that some ABIs depend on its continued existence, although library vendors would remain free to continue supplying it for their ABI needs under the zombie names policy.

Strong recommendation: take no action yet, unless this is our once-in-a-decade opportunity to break ABIs.

No change.

Weak recommendation: remove the function, and add a new entry to the [zombie.names] clause.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `uncaught_exception`, `unexpected`, and `unexpected_handler`.

~~**D.7 uncaught_exception [depr.uncaught]**~~

1. ~~The header `<exception>` has the following addition:~~

```
namespace std {
  bool uncaught_exception() noexcept;
}

bool uncaught_exception() noexcept;
```

2. ~~*Returns:* `uncaught_exceptions() > 0`.~~

Draft compatibility note for Annex C.

D.8 Old adaptable function bindings [depr.func.adaptor.binding]

The adaptable function bindings were a strong candidate for removal in C++17, but were retained only because there was no adequate replacement for users of the unary/binary negators to migrate to. That feature, `std::not_fn`, was added to C++17 to allow the migration path, with the plan to remove this obsolete facility at the first opportunity in the C++20 cycle.

There are several superior alternatives available to the classic adaptable function APIs. `std::bind` does not rely on mark-up with specific aliases in classes, and extends to an arbitrary number of parameters. Lambda expressions are often simpler to read, and integrated directly into the language.

Strong recommendation: remove this facility from C++20.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `binary_negate`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `not1`, `not2`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unary_negate`, `unexpected`, and `unexpected_handler`.
2. The following names are reserved as member types for previous standardization, and may not be used as names for object-like macros in portable code: `argument_type`, `first_argument_type`, and `second_argument_type`.

> Note that `result_type` is deliberately omitted from this list as it still has a variety of non-deprecated uses within the standard library.

~~**D.8 Old Adaptable Function Bindings [depr.func.adaptor.binding]**~~

~~**D.8.1 Weak Result Types [depr.weak.result_type]**~~

1. ~~A call wrapper (23.14.2) may have a *weak result type*. If it does, the type of its member type `result_type` is based on the type `T` of the wrapper's target object:~~
   1. ~~if `T` is a pointer to function type, `result_type` shall be a synonym for the return type of `T`;~~
   2. ~~if `T` is a pointer to member function, `result_type` shall be a synonym for the return type of `T`;~~
   3. ~~if `T` is a class type and the qualified-id `T::result_type` is valid and denotes a type (17.8.2), then `result_type` shall be a synonym for `T::result_type`;~~
   4. ~~otherwise `result_type` shall not be defined.~~

~~**D.8.2 Typedefs to Support Function Binders [depr.func.adaptor.typedefs]**~~

1. ~~To enable old function adaptors to manipulate function objects that take one or two arguments, many of the function objects in this International Standard correspondingly provide *typedef-names* `argument_type` and `result_type` for function objects that take one argument and `first_argument_type`, `second_argument_type`, and `result_type` for function objects that take two arguments.~~
2. ~~The following member names are defined in addition to names specified in Clause 23.14:~~

```
namespace std {
  template<class T> struct owner_less<shared_ptr<T> > {
    using result_type          = bool;
    using first_argument_type  = shared_ptr<T>;
    using second_argument_type = shared_ptr<T>;
  };

  template<class T> struct owner_less<weak_ptr<T> > {
    using result_type          = bool;
    using first_argument_type  = weak_ptr<T>;
    using second_argument_type = weak_ptr<T>;
  };

  template <class T> class reference_wrapper {
  public :
    using result_type          = see below; // not always defined
    using argument_type        = see below; // not always defined
    using first_argument_type  = see below; // not always defined
    using second_argument_type = see below; // not always defined
  };

  template <class T = void> struct plus {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct minus {
    using first_argument_type  = T;
    using second_argument_type = T;
    using result_type          = T;
  };

  template <class T = void> struct multiplies {
    using first_argument_type  = T;
```

```
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct divides {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct modulus {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct negate {
  using argument_type = T;
  using result_type   = T;
};

template <class T = void> struct equal_to {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct not_equal_to {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct greater {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct less {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct greater_equal {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct less_equal {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_and {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_or {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = bool;
};

template <class T = void> struct logical_not {
  using argument_type = T;
  using result_type   = bool;
};

template <class T = void> struct bit_and {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};
```

```
template <class T = void> struct bit_or {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct bit_xor {
  using first_argument_type  = T;
  using second_argument_type = T;
  using result_type          = T;
};

template <class T = void> struct bit_not {
  using argument_type = T;
  using result_type   = T;
};

template<class R, class T1>
class function<R(T1)> {
public:
  using argument_type = T1;
};

template<class R, class T1, class T2>
class function<R(T1, T2)> {
public:
  using first_argument_type  = T1;
  using second_argument_type = T2;
};
}
```

3. `reference_wrapper<T>` has a weak result type (D.7.1). If `T` is a function type, `result_type` shall be a synonym for the return type of `T`.
4. The template specialization `reference_wrapper<T>` shall define a nested type named `argument_type` as a synonym for `T1` only if the type `T` is any of the following:
   1. — a function type or a pointer to function type taking one argument of type `T1`
   2. — a pointer to member function `R T0::f cv` (where *cv* represents the member function's *cv*-qualifiers); the type `T1` is *cv* `T0*`
   3. — a class type where the *qualified-id* `T::argument_type` is valid and denotes a type (14.8.2); the type `T1` is `T::argument_type`.
5. The template instantiation `reference_wrapper<T>` shall define two nested types named `first_argument_type` and `second_argument_type` as synonyms for `T1` and `T2`, respectively, only if the type `T` is any of the following:
   1. — a function type or a pointer to function type taking two arguments of types `T1` and `T2`
   2. — a pointer to member function `R T0::f(T2) cv` (where *cv* represents the member function's *cv*-qualifiers); the type `T1` is *cv* `T0*`
   3. — a class type where the *qualified-ids* `T::first_argument_type` and `T::second_argument_type` are both valid and both denote types (14.8.2); the type `T1` is `T::first_argument_type` and the type `T2` is `T::second_argument_type`.
6. All enabled specializations `hash<Key>` of `hash` (23.14.15) provide two nested types, `result_type` and `argument_type`, which shall be synonyms for `size_t` and `Key`, respectively.
7. The forwarding call wrapper `g` returned by a call to `bind(f, bound_args...)` (23.14.11.3) shall have a weak result type (D.7.1).
8. The forwarding call wrapper `g` returned by a call to `bind<R>(f, bound_args...)` (23.14.11.3) shall have a nested type `result_type` defined as a synonym for `R`.
9. The simple call wrapper returned from a call to `mem_fn(pm)` shall have a nested type `result_type` that is a synonym for the return type of `pm` when `pm` is a pointer to member function.
10. The simple call wrapper returned from a call to `mem_fn(pm)` shall define two nested types named `argument_type` and `result_type` as synonyms for *cv* `T*` and `Ret`, respectively, when `pm` is a pointer to member function with *cv*-qualifier *cv* and taking no arguments, where `Ret` is `pm`'s return type.
11. The simple call wrapper returned from a call to `mem_fn(pm)` shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for `cv T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with *cv*-qualifier *cv* and taking one argument of type `T1`, where `Ret` is `pm`'s return type.
12. The following member names are defined in addition to names specified in Clause 26:

```
namespace std {
  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
  class map {
  public:
```

```
    class value_compare {
    public:
      using result_type         = bool;
      using first_argument_type  = value_type;
      using second_argument_type = value_type;
      }
    };
};

template <class Key, class T, class Compare = less<Key>,
          class Allocator = allocator<pair<const Key, T>>>
class multimap {
public:
    class value_compare {
    public:
      using result_type         = bool;
      using first_argument_type  = value_type;
      using second_argument_type = value_type;
    };
  };
}
```

D.8.3 Negators [depr.negators]

1. The header `<functional>` has the following additions:

```
namespace std {
  template <class Predicate> class unary_negate;
  template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate&);
  template <class Predicate> class binary_negate;
  template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate&);
}
```

2. Negators `not1` and `not2` take a unary and a binary predicate, respectively, and return their logical negations (8.3.1).

```
  template <class Predicate>
    class unary_negate {
  public:
    constexpr explicit unary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::argument_type& x) const;
    using argument_type = typename Predicate::argument_type;
    using result_type   = bool;
  };
```

3. `operator()` returns `!pred(x)`.

```
template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

4. *Returns:* `unary_negate<Predicate>(pred)`.

```
template <class Predicate>
class binary_negate {
public:
  constexpr explicit binary_negate(const Predicate& pred);
  constexpr bool operator()(const typename Predicate::first_argument_type& x,
                            const typename Predicate::second_argument_type& y) const;
  using first_argument_type = typename Predicate::first_argument_type;
  using second_argument_type = typename Predicate::second_argument_type;
  using result_type = bool;
};
```

5. `operator()` returns `!pred(x,y)`.

```
template <class Predicate>
  constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

6. *Returns:* `binary_negate<Predicate>(pred)`.

Draft compatibility note for Annex C.

Weak recommendation: Remove only the negators from C++20.

~~**D.8.3 Negators [depr.negators]**~~

1. ~~The header `<functional>` has the following additions:~~

```
namespace std {
  template <class Predicate> class unary_negate;
  template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate&);
  template <class Predicate> class binary_negate;
  template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate&);
}
```

2. ~~Negators `not1` and `not2` take a unary and a binary predicate, respectively, and return their logical negations (8.3.1).~~

```
  template <class Predicate>
    class unary_negate {
  public:
    constexpr explicit unary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::argument_type& x) const;
    using argument_type = typename Predicate::argument_type;
    using result_type   = bool;
  };
```

3. ~~`operator()` returns `!pred(x)`.~~

```
  template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

4. ~~*Returns:* `unary_negate<Predicate>(pred)`.~~

```
  template <class Predicate>
  class binary_negate {
  public:
    constexpr explicit binary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::first_argument_type& x,
                              const typename Predicate::second_argument_type& y) const;
    using first_argument_type = typename Predicate::first_argument_type;
    using second_argument_type = typename Predicate::second_argument_type;
    using result_type = bool;
  };
```

5. ~~`operator()` returns `!pred(x,y)`.~~

```
  template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

6. ~~*Returns:* `binary_negate<Predicate>(pred)`.~~

Draft compatibility note for Annex C.

D.9 The default allocator [depr.default.allocator]

One surprising issue turned up with an implementation that eagerly removed the deprecated names. It turns out that there are platforms where `size_t` and `ptrdiff_t` are not signed/unsigned variations on the same underlying type, so relying on the default type computation through `allocator_traits` produces ABI incompatibilities. It appears reasonably safe to remove the other members once direct use has diminished. This would be a noticable compatibility hurdle for containers trying to retain compatibility with both C++03 code and C++20, due to the lack of `allocator_traits` in that earlier standard. However, most of a decade will have passed since the publication of C++11 by the time C++20 is published, and that may be deemed sufficient time to address compatibility concerns.

`allocator<void>` does not serve a useful compatibility purpose, and should safely be removed. There are no special names to be reserved as zombies, as all removed identifiers continue to be used throughout the library.

Strong recommendation: Undeprecate `std::allocator<T>::size_type` and `std::allocator<T>::difference_type`, and remove the remaining deprecated parts from the C++20 standard:

23.10.9 The default allocator [default.allocator]

```
namespace std {
  template <class T> class allocator {
  public:
    using value_type      = T;
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using is_always_equal = true_type;
    using propagate_on_container_move_assignment = true_type;

    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template  allocator(const allocator<U>&) noexcept;
    ~allocator();

    T* allocate(size_t n);
    void deallocate(T* p, size_t n);
  };
}
```

~~**D.9 The default allocator [depr.default.allocator]**~~

1. ~~The following members are defined in addition to those specified in Clause 20:~~

```
namespace std {
  template <> class allocator<void> {
  public:
    using value_type    = void;
    using pointer       = void*;
    using const_pointer = const void*;
    // reference-to-void members are impossible.
    template <class U> struct rebind { using other = allocator<U>; };
  };

  template <class T> class allocator {
  public:
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using pointer         = T*;
    using const_pointer   = const T*;
    using reference       = T&;
    using const_reference = const T&;
    template <class U> struct rebind { using other = allocator<U>; };

    T* address(reference x) const noexcept;
    const T* address(const_reference x) const noexcept;

    T* allocate(size_t, const void* hint);

    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);

    size_t max_size() const noexcept;
  };
}

T* address(reference x) const noexcept;
const T* address(const_reference x) const noexcept;
```

2. ~~*Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.~~

```
T* allocate(size_t, const void* hint);
```

3. ~~*Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (3.11).~~
4. ~~*Remark:* the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called.~~
5. ~~*Throws:* `bad_alloc` if the storage cannot be obtained.~~

```
template <class U, class... Args>
  void construct(U* p, Args&&... args);
```

6. ~~*Effects:* `::new((void *)p) U(std::forward<Args>(args)...)`~~

```
template <class U>
```

~~void destroy(U* p);~~

7. ~~*Effects:* `p->~U()`~~

~~size_type max_size() const noexcept;~~

8. ~~*Returns:* The largest value `N` for which the call `allocate(N)` might succeed.~~

Draft compatibility note for Annex C.

Weak recommendation: Undeprecate `std::allocator<T>::size_type` and `std::allocator<T>::difference_type`, and remove (just) `std::allocator<void>`:

23.10.9 The default allocator [default.allocator]

```
namespace std {
  template <class T> class allocator {
  public:
    using value_type      = T;
    using size_type       = size_t;
    using difference_type = ptrdiff_t;
    using is_always_equal = true_type;
    using propagate_on_container_move_assignment = true_type;

    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template  allocator(const allocator<U>&) noexcept;
    ~allocator();

    T* allocate(size_t n);
    void deallocate(T* p, size_t n);
  };
}
```

D.9 The default allocator [depr.default.allocator]

1. The following members are defined in addition to those specified in Clause 20:

```
namespace std {
  ~~template <> class allocator<void> {~~
  ~~public:~~
    ~~using value_type    = void;~~
    ~~using pointer       = void*;~~
    ~~using const_pointer = const void*;~~
    ~~// reference-to-void members are impossible.~~
    ~~template <class U> struct rebind { using other = allocator<U>; };~~
  ~~};~~

  template <class T> class allocator {
  public:
    ~~using size_type       = size_t;~~
    ~~using difference_type = ptrdiff_t;~~
    using pointer         = T*;
    using const_pointer   = const T*;
    using reference       = T&;
    using const_reference = const T&;
    template <class U> struct rebind { using other = allocator<U>; };

    T* address(reference x) const noexcept;
    const T* address(const_reference x) const noexcept;

    T* allocate(size_t, const void* hint);

    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);

    size_t max_size() const noexcept;
  };
}
```

Draft compatibility note for Annex C.

D.10 Raw storage iterator [depr.storage.iterator]

`raw_storage_iterator` is a limited facility with several shortcomings that are unlikely to be addressed. The most obvious is that in its intended use in an algorithm, there is no clear way to identify how many items have been constructed and would be cleaned up if a constructor throws. Such a fundamentally unsafe type has no place in a modern standard library.

Additional concerns include that it does not support allocators, and does not call `allocator_traits::construct` to initialize elements, making it unsuitable as an implementation detail for the majority of containers.

To continue as an integral part of the C++ standard library it should acquire external deduction guides to deduce `T` from the `value_type` of the passed `OutputIterator`, but it does not seem worth the effort to revive.

The class template may live on a while longer as a zombie name, allowing vendors to wean customers off at their own pace, but this class no longer belongs in the C++ Standard Library.

Strong recommendation: remove the deprecated iterator from C++20.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `raw_storage_iterator`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.

~~**D.10 Raw storage iterator [depr.storage.iterator]**~~

~~The header `<memory>` has the following addition:~~

```
namespace std {
  template <class OutputIterator, class T>
  class raw_storage_iterator {
  public:
    using iterator_category = output_iterator_tag;
    using value_type        = void;
    using difference_type   = void;
    using pointer           = void;
    using reference         = void;

    explicit raw_storage_iterator(OutputIterator x);

    raw_storage_iterator& operator*();
    raw_storage_iterator& operator=(const T& element);
    raw_storage_iterator& operator=(T&& element);
    raw_storage_iterator& operator++();
    raw_storage_iterator  operator++(int);
    OutputIterator base() const;
  };
}
```

1. ~~`raw_storage_iterator` is provided to enable algorithms to store their results into uninitialized memory. The template parameter `OutputIterator` is required to have its `operator*` return an object for which `operator&` is defined and returns a pointer to `T`, and is also required to satisfy the requirements of an output iterator (24.2.4).~~

   ~~`explicit raw_storage_iterator(OutputIterator x);`~~

2. ~~*Effects:* Initializes the iterator to point to the same value to which `x` points.~~

   ~~`raw_storage_iterator& operator*();`~~

3. ~~*Returns:* `*this`~~

   ~~`raw_storage_iterator& operator=(const T& element);`~~

4. ~~*Requires:* `T` shall be `CopyConstructible`.~~
5. ~~*Effects:* Constructs a value from `element` at the location to which the iterator points.~~
6. ~~*Returns:* A reference to the iterator.~~

   ~~`raw_storage_iterator& operator=(T&& element);`~~

7. ~~*Requires:* `T` shall be `MoveConstructible`.~~
8. ~~*Effects:* Constructs a value from `std::move(element)` at the location to which the iterator points.~~
9. ~~*Returns:* A reference to the iterator.~~

   ~~`raw_storage_iterator& operator++();`~~

10. ~~*Effects:* Pre-increment: advances the iterator and returns a reference to the updated iterator.~~

    ~~`raw_storage_iterator operator++(int);`~~

11. ~~*Effects:* Post-increment: advances the iterator and returns the old value of the iterator.~~

    ~~`OutputIterator base() const;`~~

12. ~~*Returns:* An iterator of type `OutputIterator` that points to the same value as `*this` points to.~~

Draft compatibility note for Annex C.

Weak recommendation: take no action.

No change.

D.11 Temporary buffers [depr.temporary.buffer]

Strong recommendation: remove the awkward API.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_temporary_buffer`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `return_temporary_buffer`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.

~~D.11 Temporary buffers [depr.temporary.buffer]~~

1. ~~The header `<memory>` has the following additional functions:~~

```
namespace std {
  // 20.9.11, temporary buffers:
  template <class T>
    pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
  template <class T>
    void return_temporary_buffer(T* p);
}
```

```
template <class T>
  pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

2. ~~*Effects:* Obtains a pointer to uninitialized, contiguous storage for *N* adjacent objects of type `T`, for some non-negative number *N*. It is implementation-defined whether over-aligned types are supported (3.11).~~
3. ~~*Remarks:* Calling `get_temporary_buffer` with a positive number `n` is a non-binding request to return storage for `n` objects of type `T`. In this case, an implementation is permitted to return instead storage for a non-negative number *N* of such objects, where *N* `!= n` (including *N* `== 0`). [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations of its memory management. - *end note*]~~
4. ~~*Returns:* If `n <= 0` or if no storage could be obtained, returns a pair `P` such that `P.first` is a null pointer value and `P.second == 0`; otherwise returns a pair `P` such that `P.first` refers to the address of the uninitialized storage and `P.second` refers to its capacity *N* (in the units of `sizeof(T)`).~~

```
template <class T> void return_temporary_buffer(T* p);
```

1. ~~*Effects:* Deallocates the storage referenced by `p`.~~
2. ~~*Requires:* `p` shall be a pointer value returned by an earlier call to `get_temporary_buffer` that has not been invalidated by an intervening call to `return_temporary_buffer(T*)`.~~
3. ~~*Throws:* Nothing.~~

Draft compatibility note for Annex C.

Weak recommendation: add sufficient facilities for safe use that the facility can be undeprecated.

Details are deferred to a follow-up paper, if desired.

D.12 Deprecated type traits [depr.meta.types]

Strong recommendation: Remove the traits that can live on as zombies.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `is_literal_type`, `is_literal_type_v`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `result_of`, `result_of_t`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.

~~D.12 Deprecated Type Traits [depr.meta.type]~~

1. ~~The header `<type_traits>` has the following additions:~~

```
namespace std {
  template <class T> struct is_literal_type;

  template <class T> constexpr bool is_literal_type_v = is_literal_type<T>::value;

  template <class> struct result_of; // not defined
  template <class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;

  template <class T> using result_of_t = typename result_of<T>::type;
}
```

2. ~~*Requires:* For `is_literal_type`, `remove_all_extents_t<T>` shall be a complete type or *cv* `void`. For `result_of<Fn(ArgTypes...)>`, `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.~~
3. ~~`is_literal_type` is a `UnaryTypeTrait` (23.15.1) with a base characteristic of `true_type` if `T` is a literal type (3.9), and `false_type` otherwise. The partial specialization `result_of<Fn(ArgTypes...)>` is a `TransformationTrait` whose member typedef `type` is defined if and only if `invoke_result<Fn, ArgTypes...>::type` is defined. If `type` is defined, it names the same type as `invoke_result_t<Fn, ArgTypes...>`.~~
4. ~~The behavior of a program that adds specializations for `is_literal_type` or `is_literal_type_v` is undefined.~~

Draft compatibility note for Annex C.

Weak recommendation: take no action at this time.

No change.

D.13 Deprecated iterator primitives [depr.iterator.primitives]

Strong recommendation: enhance `iterator_traits` with implicit deductions of each nested name, to better facilitate removal of this feature.

Details are deferred to a follow-up paper, if desired, and apply the edit below.

Weak recommendation: remove this awkward class template, whether or not an improved deduction facility becomes available.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `iterator`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.

~~D.13 Deprecated iterator primitives [depr.iterator.primitives]~~

~~D.13.1 Basic iterator [depr.iterator.basic]~~

1. ~~The header `<iterator>` has the following addition:~~

```
namespace std {
  template<class Category, class T, class Distance = ptrdiff_t,
    class Pointer = T*, class Reference = T&>
  struct iterator {
    using iterator_category = Category;
    using value_type        = T;
```

```
    using difference_type   = Distance;
    using pointer           = Pointer;
    using reference         = Reference;
  };
}
```

2. The `iterator` template may be used as a base class to ease the definition of required types for new iterators.
3. [*Note:* If the new iterator type is a class template, then these aliases will not be visible from within the iterator class's template definition, but only to callers of that class - *end note*]
4. [*Example:* If a C++ program wants to define a bidirectional iterator for some data structure containing `double` and such that it works on a large memory model of the implementation, it can do so with:

```
class MyIterator :
  public iterator<bidirectional_iterator_tag, double, long, T*, T&> {
  // code implementing ++, etc.
};
```

— end example]

Draft compatibility note for Annex C.

D.14 Deprecated `shared_ptr` observers [depr.util.smartptr.shared.obs]

Strong recommendation: Remove misleading function, as `use_count` remains for single-threaded use.

D.13 Deprecated shared_ptr observers [depr.util.smartptr.shared.obs]

1. The following member is defined in addition to those members specified in 23.11.2.2:

```
namespace std {
  template class shared_ptr {
  public:
    bool unique() const noexcept;
  };
}
```

```
bool unique() const noexcept;
```

2. *Returns:* `use_count() == 1`.

Draft compatibility note for Annex C.

Weak recommendation: take no action yet.

No change.

D.15 Standard code conversion facets [depr.locale.stdcvt]

Strong recommendation: Remove the header `<codecvt>`; retain all names, including header name, as zombies.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `codecvt_mode`, `codecvt_utf16`, `codecvt_utf8`, `codecvt_utf8_utf16`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `consume_header`, `generate_header`, `get_unexpected`, `little_endian`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.
2. The header name `<codecvt>` is reserved for previous standardization.

D.14 Standard code conversion facets [depr.locale.stdcvt]

D.14.1 Header `<codecvt>` synopsis [depr.codecvt.syn]

1. The header `<codecvt>` provides code conversion facets for various character encodings.

```
namespace std {
  enum codecvt_mode {
    consume_header = 4,
```

```
    generate_header = 2,
    little_endian = 1
};

template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
class codecvt_utf8
    : public codecvt<Elem, char, mbstate_t> {
public:
    explicit codecvt_utf8(size_t refs = 0);
    ~codecvt_utf8();
};

template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
class codecvt_utf16
    : public codecvt<Elem, char, mbstate_t> {
public:
    explicit codecvt_utf16(size_t refs = 0);
    ~codecvt_utf16();
};

template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
class codecvt_utf8_utf16
    : public codecvt<Elem, char, mbstate_t> {
public:
    explicit codecvt_utf8_utf16(size_t refs = 0);
    ~codecvt_utf8_utf16();
};
}
```

D.14.2 Requirements [depr.locale.stdcvt.req]

1. For each of the three code conversion facets `codecvt_utf8`, `codecvt_utf16`, and `codecvt_utf8_utf16`:
   1. — `Elem` is the wide-character type, such as `wchar_t`, `char16_t`, or `char32_t`.
   2. — `Maxcode` is the largest wide-character code that the facet will read or write without reporting a conversion error.
   3. — If `(Mode & consume_header)`, the facet shall consume an initial header sequence, if present, when reading a multibyte sequence to determine the endianness of the subsequent multibyte sequence to be read.
   4. — If `(Mode & generate_header)`, the facet shall generate an initial header sequence when writing a multibyte sequence to advertise the endianness of the subsequent multibyte sequence to be written.
   5. — If `(Mode & little_endian)`, the facet shall generate a multibyte sequence in little-endian order, as opposed to the default big-endian order.
2. For the facet `codecvt_utf8`:
   1. — The facet shall convert between UTF-8 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
   2. — Endianness shall not affect how multibyte sequences are read or written.
   3. — The multibyte sequences may be written as either a text or a binary file.
3. For the facet `codecvt_utf16`:
   1. — The facet shall convert between UTF-16 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
   2. — Multibyte sequences shall be read or written according to the `Mode` flag, as set out above.
   3. — The multibyte sequences may be written only as a binary file. Attempting to write to a text file produces undefined behavior.
4. For the facet `codecvt_utf8_utf16`:
   1. — The facet shall convert between UTF-8 multibyte sequences and UTF-16 (one or two 16-bit codes) within the program.
   2. — Endianness shall not affect how multibyte sequences are read or written.
   3. — The multibyte sequences may be written as either a text or a binary file.

   See also: ISO/IEC 10646-1:1993.

Draft compatibility note for Annex C.

Weak recommendation: take no action yet.

No change.

D.16 Deprecated character conversions [depr.conversions]

Strong recommendation: Remove this facility from the standard at the earliest opportunity.

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unexpected`, ~~and~~ `unexpected_handler`, `wbuffer_convert`, and `wstring_convert`.
2. The following names are reserved as member functions for previous standardization, and may not be used as names for function-like macros in portable code: `converted`, `from_bytes`, and `to_bytes`.

~~**D.16.1 Class template wstring_convert [depr.conversions.string]**~~

1. ~~Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:~~

```
wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

~~*— end example*]~~

```
namespace std {
template <class Codecvt, class Elem = wchar_t,
          class Wide_alloc = allocator<Elem>,
          class Byte_alloc = allocator<char>>
  class wstring_convert {
  public:
    using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
    using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;
    using state_type  = typename Codecvt::state_type;
    using int_type    = typename wide_string::traits_type::int_type;

    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                             const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);
    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);
    size_t converted() const noexcept;
    state_type state() const;
  private:
    byte_string byte_err_string;
    wide_string wide_err_string;
    Codecvt* cvtptr;
    state_type cvtstate;
    size_t cvtcount;
  };
}
```

2. ~~The class template describes an object that controls conversions between wide string objects of class `basic_string<Elem, char_traits<Elem>, Wide_alloc>` and byte string objects of class `basic_string<char, char_traits<char>, Byte_alloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.~~

3. An object of this class template stores:
    1. — `byte_err_string` — a byte string to display on errors
    2. — `wide_err_string` — a wide string to display on errors
    3. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)
    4. — `cvtstate` — a conversion state object
    5. — `cvtcount` — a conversion count

    ```
    using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
    ```

4. The type shall be a synonym for `basic_string<char, char_traits<char>, Byte_alloc>`

    ```
    size_t converted() const noexcept;
    ```

5. *Returns:* `cvtcount`.

    ```
    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);
    ```

6. *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in str to a wide string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a wide string.
7. In all cases:
    1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
    2. — The number of input elements successfully converted shall be stored in `cvtcount`.
8. *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `range_error`.

    ```
    using int_type = typename wide_string::traits_type::int_type;
    ```

9. The type shall be a synonym for `wide_string::traits_type::int_type`.

    ```
    state_type state() const;
    ```

10. returns `cvtstate`.

    ```
    using state_type = typename Codecvt::state_type;
    ```

11. The type shall be a synonym for `Codecvt::state_type`.

    ```
    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);
    ```

12. *Effects:* The first member function shall convert the single-element sequence `wchar` to a byte string. The second member function shall convert the null-terminated sequence beginning at `wptr` to a byte string. The third member function shall convert the sequence stored in wstr to a byte string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a byte string.
13. In all cases:
    1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
    2. — The number of input elements successfully converted shall be stored in `cvtcount`.
14. *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class `range_error`.

    ```
    using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;
    ```

15. The type shall be a synonym for `basic_string<Elem, char_traits<Elem>, Wide_alloc>`.

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());
```

16. *Requires:* For the first and second constructors, `pcvt != nullptr`.
17. *Effects:* The first constructor shall store `pcvt` in `cvtptr` and default values in `cvtstate`, `byte_err_string`, and `wide_err_string`. The second constructor shall store `pcvt` in `cvtptr`, `state` in `cvtstate`, and default values in `byte_err_string` and `wide_err_string`; moreover the stored state shall be retained between calls to `from_bytes` and `to_bytes`. The third constructor shall store `new Codecvt` in `cvtptr`, `state_type()` in `cvtstate`, `byte_err` in `byte_err_string`, and `wide_err` in `wide_err_string`.

```
~wstring_convert();
```

18. *Effects:* The destructor shall delete `cvtptr`.

D.16.2 Class template wbuffer_convert [depr.conversions.buffer]

1. Class template `wbuffer_convert` looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template `wstring_convert`, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

```
namespace std {
template <class Codecvt, class Elem = wchar_t, class Tr = char_traits<Elem>>
  class wbuffer_convert
    : public basic_streambuf<Elem, Tr> {
  public:
    using state_type = typename Codecvt::state_type;

    explicit wbuffer_convert(streambuf* bytebuf = 0,
                             Codecvt* pcvt = new Codecvt,
                             state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    streambuf* rdbuf() const;
    streambuf* rdbuf(streambuf* bytebuf);

    state_type state() const;

  private:
    streambuf* bufptr;          // exposition only
    Codecvt* cvtptr;            // exposition only
    state_type cvtstate;        // exposition only
  };
}
```

2. The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
    1. — `bufptr` — a pointer to its underlying byte stream buffer
    2. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)
    3. — `cvtstate` — a conversion state object

```
state_type state() const;
```

4. *Returns:* `cvtstate`.

```
streambuf* rdbuf() const;
```

5. *Returns:* `bufptr`.

```
streambuf* rdbuf(streambuf* bytebuf);
```

6. *Effects:* Stores `bytebuf` in `bufptr`.
7. *Returns:* The previous value of `bufptr`.

```
using state_type = typename Codecvt::state_type;
```

8. The type shall be a synonym for `Codecvt::state_type`.

```
explicit wbuffer_convert(streambuf* bytebuf = 0,
    Codecvt* pcvt = new Codecvt, state_type state = state_type());
```

9. *Requires:* `pcvt != nullptr`.
10. *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`.

```
~wbuffer_convert();
```

11. *Effects:* The destructor shall delete `cvtptr`.

Draft compatibility note for Annex C.

Weak recommendation: take no action yet.

No change.

4 Other Directions

While practicing good housekeeping and clearing out Annex D for each release may be the preferred option, there are other approaches that may be taken.

Do Nothing

One approach, epitomised in the Java language, is that deprecated features are discouraged for future use, but guaranteed to remain available forever, and just accumulate.

This approach is rejected by this paper for a number of reasons. First, C++ has been relatively successful in actually removing its deprecated features in the past, a tradition we would like to continue. It also undercuts the available-forever rationale, as it is not a guarantee we have given before.

A second concern is that we do not want to pay a cost to maintain deprecated components forever - restricting growth of the language for compatibility with deprecated features, or having to review the whole of Annex D and upgrade components for every new language release, in order to keep up with subtle shifts in the core language.

Undeprecate

If something is deprecated, but later (re)discovered to have value, then it could be revitalized and restored to the main standard. For example, this is exactly what happened to static function declarations when the unnamed namespace was given internal linkage - it is merely the classical way to say the same thing, and often clearer to write.

This may be a consideration for long-term deprecated features that don't appear to be going anywhere, such as the `strstream` facility, or the C headers. It may be appropriate to find them a home in the regular standard, and this is called out in the specific reviews of each facility above.

5 Formal Wording

For now, this section does not contain any wording for feature removal or undeprecation, as the proposed drafting for (re)moving each facility is attached to the subsection reviewing those features. Once the relevant evolution groups have agreed which removals should proceed, the wording will be consolidated here as a simpler direction for the project editor(s) to apply.

Below is a minimal set of edits that fix unreported issues with the current standard, that bring it much closer to its original intent wrt some of the features explored above. These changes are proposed regardless of the acceptance or rejection of any (or all) of the suggestions above.

Add missing zombies from prior standards:

20.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `auto_ptr_ref`, `binary_function`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `const_mem_fun1_ref_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun_t`, `get_unexpected`, `gets`, `mem_fun1_ref_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun_ref`, `mem_fun_t`, `mem_fun`, `pointer_to_binary_function`, `pointer_to_unary_function`, `ptr_fun`, `random_shuffle`, `set_unexpected`, `unary_function`, `unexpected`, and `unexpected_handler`.
2. The following names are reserved as member types for previous standardization, and may not be used as names for object-like macros in portable code: `io_state`, `open_mode`, and `seek_dir`
3. The following name is reserved as a member function for previous standardization, and may not be used as names for function-like macros in portable code: `stossc`.

Provide default copy operations for the following classes, so that they no longer rely on deprecated copy constructor/assignment generation, even if the feature is not removed:

- `bitset<N>::reference` - assign, no copy ctor
- `allocator` - copy ctor, no assignment
- `pmr::memory_resource` - dtor only, no copy/assign
- `vector<bool>::reference` - assign, no copy ctor
- `istream_iterator` - copy ctor, no assignment
- `ostream_iterator` - copy ctor, no assignment
- `istreambuf_iterator` - copy ctor, no assignment
- `complex<float>` - assign, no copy ctor
- `complex<double>` - assign, no copy ctor
- `complex<long double>` - assign, no copy ctor
- `ios_base::Init` - dtor only, no copy/assign

23.9.2 Class template bitset [template.bitset]

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference:
    class reference {
      friend class bitset;
      reference() noexcept;
    public:
      reference(const reference&) = default;
      ~reference() noexcept;
      reference& operator=(bool x) noexcept;             // for b[i] = x;
      reference& operator=(const reference&) noexcept;   // for b[i] = b[j];
      bool operator~() const noexcept;                   // flips the bit
      operator bool() const noexcept;                    // for x = b[i];
      reference& flip() noexcept;                        // for b[i].flip();
    };

    // 23.9.2.1, constructors
    ...
};
```

Standard library destructors are implicitly `noexcept`, so it is actually confusing to make it explicit here and nowhere else. Removed as a drive-by fix.

23.10.9 The default allocator [default.allocator]

```
namespace std {
  template <class T> class allocator {
  public:
    using value_type      = T;
    using is_always_equal = true_type;
    using propagate_on_container_move_assignment = true_type;

    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template  allocator(const allocator<U>&) noexcept;
    ~allocator();
    allocator& operator=(const allocator&) noexcept = default;

    T* allocate(size_t n);
    void deallocate(T* p, size_t n);
  };
}
```

23.12.2 Class memory_resource [mem.res.class]

```
namespace std {
  class memory_resource {
    static constexpr size_t max_align = alignof(max_align_t); // exposition only

  public:
    memory_resource(const memory_resource&) = default;
    virtual ~memory_resource();

    memory_resource&     operator=(const memory_resource&) = default;

    void* allocate(size_t bytes, size_t alignment = max_align);
    void deallocate(void* p, size_t bytes, size_t alignment = max_align);

    bool is_equal(const memory_resource& other) const noexcept;

  private:
    virtual void* do_allocate(size_t bytes, size_t alignment) = 0;
    virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;

    virtual bool do_is_equal(const memory_resource& other) const noexcept = 0;
  };
}
```

Although copy operations are defaulted here for compatibility with the implicit declarations of C++17, it would be consistent with the original design to actually delete them, and provide a protected default constructor. All the derived implementations in the standard library delete both copy constructor and copy-assignment, and in doing so, inhibit the move operations too.

26.3.12 Class `vector<bool>`

```
namespace std {
  template <class Allocator>
  class vector<bool, Allocator> {
  public:
    // types:
    ...

    // bit reference:
    class reference {
      friend class vector;
      reference() noexcept;

    public:
      reference(const reference&) = default;
      ~reference();
      operator bool() const noexcept;
      reference& operator=(const bool x) noexcept;
      reference& operator=(const reference& x) noexcept;
      void flip() noexcept; // flips the bit
    };

    // construct/copy/destroy:
    ...
  };
}
```

27.6.1 Class template istream_iterator [istream.iterator]

```
namespace std {
  template <class T, class charT = char, class traits = char_traits<charT>,
      class Distance = ptrdiff_t>
  class istream_iterator {
  public:
    using iterator_category = input_iterator_tag;
    using value_type        = T;
    using difference_type   = Distance;
    using pointer           = const T*;
    using reference         = const T&;
    using char_type         = charT;
    using traits_type       = traits;
    using istream_type      = basic_istream<charT,traits>;

    constexpr istream_iterator();
    istream_iterator(istream_type& s);
```

```
    istream_iterator(const istream_iterator& x) = default;
    ~istream_iterator() = default;

    istream_iterator& (istream_type& s); = default;
    const T& operator*() const;
    const T* operator->() const;
    istream_iterator& operator++();
    istream_iterator  operator++(int);
  private:
    basic_istream<charT,traits>* in_stream; // exposition only
    T value;                                // exposition only
  };
}
```

27.6.2 Class template ostream_iterator [ostream.iterator]

```
namespace std {
  template <class charT = char, class traits = char_traits<charT>>
  class ostream_iterator {
  public:
    using iterator_category = ouput_iterator_tag;
    using value_type        = void;
    using difference_type   = void;
    using pointer           = void;
    using reference         = void;
    using char_type         = charT;
    using traits_type       = traits;
    using ostream_type      = basic_istream<charT,traits>;

    ostream_iterator(ostream_type& s);
    ostream_iterator(ostream_type& s, const charT* delimiter);
    ostream_iterator(const ostream_iterator& x);
    ~ostream_iterator();

    ostream_iterator& operator=(const ostream_iterator& x);
    ostream_iterator& operator=(const T& value);
    ostream_iterator& operator*();
    ostream_iterator& operator++();
    ostream_iterator& operator++(int);
  private:
    basic_ostream<charT,traits>* out_stream;  // exposition only
    const charT* delim;                       // exposition only
  };
}
```

27.6.3 Class template istreambuf_iterator [istreambuf.iterator]

```
namespace std {
  template <class T, class charT = char, class traits = char_traits<charT>>
  class ostream_iterator {
  public:
    using iterator_category = ouput_iterator_tag;
    using value_type        = void;
    using difference_type   = void;
    using pointer           = void;
    using reference         = void;
    using char_type         = charT;
    using traits_type       = traits;
    using ostream_type      = basic_istream<charT,traits>;

    class proxy;                          // exposition only

    constexpr istreambuf_iterator() noexcept;
    istreambuf_iterator(const istreambuf_iterator&) noexcept = default;
    ~istreambuf_iterator() = default;
    istreambuf_iterator(istream_type& s) noexcept;
    istreambuf_iterator(streambuf_type* s) noexcept;
    istreambuf_iterator(const proxy& p) noexcept;

    istreambuf_iterator& operator=(const istreambuf_iterator&) noexcept = default;
    charT operator*() const;
    pointer operator->() const;
    istreambuf_iterator& operator++();
    proxy operator++(int);
    bool equal(const istreambuf_iterator& b) const;
  private:
    streambuf_type* sbuf_;               // exposition only
```

```
  };
}
```

29.5.3 complex specializations [complex.special]

```
namespace std {
  template<> class complex<float> {
  public:
    using value_type = float;

    complex(const complex&) = default;
    constexpr complex(float re = 0.0f, float im = 0.0f);
    constexpr explicit complex(const complex<double>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr float real() const;
    void real(float);
    constexpr float imag() const;
    void imag(float);

    complex<float>& operator= (float);
    complex<float>& operator+=(float);
    complex<float>& operator-=(float);
    complex<float>& operator*=(float);
    complex<float>& operator/=(float);

    complex<float>& operator=(const complex<float>&);
    template<class X> complex<float>& operator= (const complex<X>&);
    template<class X> complex<float>& operator+=(const complex<X>&);
    template<class X> complex<float>& operator-=(const complex<X>&);
    template<class X> complex<float>& operator*=(const complex<X>&);
    template<class X> complex<float>& operator/=(const complex<X>&);
  };

  template<> class complex<double> {
  public:
    using value_type = double;

    complex(const complex&) = default;
    constexpr complex(double re = 0.0, double im = 0.0);
    constexpr complex(const complex<float>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr double real() const;
    void real(double);
    constexpr double imag() const;
    void imag(double);

    complex<double>& operator= (double);
    complex<double>& operator+=(double);
    complex<double>& operator-=(double);
    complex<double>& operator*=(double);
    complex<double>& operator/=(double);

    complex<double>& operator=(const complex<double>&);
    template<class X> complex<double>& operator= (const complex<X>&);
    template<class X> complex<double>& operator+=(const complex<X>&);
    template<class X> complex<double>& operator-=(const complex<X>&);
    template<class X> complex<double>& operator*=(const complex<X>&);
    template<class X> complex<double>& operator/=(const complex<X>&);
  };

  template<> class complex<long double> {
  public:
    using value_type = long double;

    complex(const complex&) = default;
    constexpr complex(long double re = 0.0L, long double im = 0.0L);
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&);

    constexpr long double real() const;
    void real(long double);
    constexpr long double imag() const;
    void imag(long double);

    complex<long double>& operator=(const complex<long double>&);
    complex<long double>& operator= (long double);
```

```
    complex<long double>& operator+=(long double);
    complex<long double>& operator-=(long double);
    complex<long double>& operator*=(long double);
    complex<long double>& operator/=(long double);

    template<class X> complex<long double>& operator= (const complex<X>&);
    template<class X> complex<long double>& operator+=(const complex<X>&);
    template<class X> complex<long double>& operator-=(const complex<X>&);
    template<class X> complex<long double>& operator*=(const complex<X>&);
    template<class X> complex<long double>& operator/=(const complex<X>&);
  };
}
```

30.5.3.1.6 Class ios_base::Init [ios::Init]

```
namespace std {
  class ios_base::Init {
  public:
    Init();
    Init(const Init&) = default;
    ~Init();

    Init& operator=(const Init&) = default;
  private:
    static int init_cnt; // exposition only
  };
}
```

> In this case, implicitly deleting the copy operations may well be the right answer, although we propose the current edit as no change of existing semantics.

6 Acknowledements

Thanks to everyone who worked on flagging these facilities for deprecation, let's take the next step!

7 References

- N3201 Moving right along, Bjarne Stroustrup
- N3203 Tightening the conditions for generating implicit moves, Jens Maurer
- N4086 Removing trigraphs??!, Richard Smith
- N4190 Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- N4259 Wording for std::uncaught_exceptions, Herb Sutter
- P0001R1 Remove Deprecated Use of the `register` Keyword, Alisdair Meredith
- P0002R1 Remove Deprecated `operator++(bool)`, Alisdair Meredith
- P0003R5 Remove Deprecated Exception Specifications from C++17, Alisdair Meredith
- P0004R1 Remove Deprecated iostreams aliases, Alisdair Meredith
- P0005R4 Adopt 'not_fn' from Library Fundamentals 2 for C++17, Alisdair Meredith, Stephan T. Lavavej, Tomasz Kamiński
- P0174R2 Deprecating Vestigial Library Parts in C++17, Alisdair Meredith
- P0386R2 Inline Variables, Hal Finkel and Richard Smith
- P0521R0 Proposed Resolution for CA 14 (shared_ptr use_count/unique), Stephan T. Lavavej
- P0618R0 Deprecating `<codecvt>`, Alisdair Meredith